

DAY 9 TECHNICAL REPORT

Sorting Performance Comparison

Asymptotic Benchmarking & Empirical Complexity Evaluation

TRAINEE: Benatic Rithan B

MENTOR: Rathees G

INSTITUTION: KIT, Coimbatore

DEPARTMENT: CSE

DATE: 18 June 2026

1. EXECUTIVE SUMMARY

This technical report documents the implementation and empirical validation of a high-performance evaluation suite developed during Day 9 of the Python Backend Development Internship. The design objectives focused on evaluating and benchmarking core classical sorting algorithms—categorized by their asymptotic upper bounds into quadratic $O(n^2)$ variants (Bubble Sort, Selection Sort) and logarithmic $O(n \log n)$ variants (Merge Sort, Quick Sort)—under distinct input data configurations. The engineered application tracks exact execution deltas, uncovers worst-case runtime vulnerabilities, and builds an analytical foundation for choosing algorithms based on structural constraints.

2. DATA STRUCTURE SELECTION & COMPLEXITY ARCHITECTURE

To systematically orchestrate the comparative suite without runtime infrastructure artifacts, primitive data structures and object schemas were assigned specialized roles based on their computational footprints:

- **Dictionaries (Hash Maps):** Chosen as the core runtime registry mapping arbitrary algorithm identifiers to their respective functional definitions to execute dynamic benchmark routing loops in constant time, $O(1)$.
- **Tuples (Immutable Sequences):** Utilized to construct permanent, structured performance metric points (e.g., algorithm name, configuration profile, array size, execution duration) to protect raw experiment output from accidental in-flight alteration.
- **Lists (Dynamic Arrays):** Employed as contiguous mutable segments to hold structural data variations, allowing continuous mutation testing across varying sizes.
- **Sets (Hash Tables):** Applied to log unique data scale dimensions checked during experimental loops, enabling instant duplication checks at $O(1)$ checkpoints.

3. STRUCTURAL MAPPING MATRIX

ALGORITHM	COMPLEXITY (BEST / AVG / WORST)	SPACE COMPLEXITY	PRIMARY OPERATIONAL ADVANTAGE & BEHAVIOR
Bubble Sort	$O(n) / O(n^2) / O(n^2)$	$O(1)$	Local adjacent element swaps; features adaptive early termination on pre-sorted arrays.
Selection Sort	$O(n^2) / O(n^2) / O(n^2)$	$O(1)$	Continuous linear scans to isolate minimum values; execution path is entirely invariant to data order.
Merge Sort	$O(n \log n)$ across all cases	$O(n)$	Stable Divide-and-Conquer strategy providing guaranteed timing boundaries at the expense of external allocation.
Quick Sort	$O(n \log n) / O(n \log n) / O(n^2)$	$O(\log n)$	In-place partitioning around highly volatile pivot selections; highly performant on average but vulnerable to worst-case degradation.

4. TECHNICAL CODE IMPLEMENTATION

The code segment below showcases the integration of multiple collection structures to coordinate execution benchmarks safely across varying array configurations using high-resolution timers:

```
import time
import random

class SortingBenchmarkManager:
    def __init__(self):
        # Dictionary for O(1) lookups: algorithm_name -> function_pointer
        self.algorithms = {} # Registry

        # Set for tracking unique array sizes tested
        self.tested_sizes = set() # Unique Nodes

        # List of tuples for historical execution audit logs
        self.performance_history = [] # Chronological Ledger

    def register_algorithm(self, name, function_ptr):
        self.algorithms[name] = function_ptr

    def run_benchmark(self, dataset_type, size):
        # Generate target dataset configurations based on targeted profiles
        if dataset_type == "RANDOM":
            data = [random.randint(1, 10000) for _ in range(size)]
        elif dataset_type == "SORTED":
            data = list(range(size))
```

```

elif dataset_type == "REVERSED":
    data = list(range(size, 0, -1))
else:
    raise ValueError(f"Dataset type {dataset_type} is not supported.")

self.tested_sizes.add(size)

# Execute performance tracking safely across isolation boundaries
for name, algorithm_func in self.algorithms.items():
    data_copy = list(data) # Maintain integrity across subsequent trials

    start_time = time.perf_counter()
    algorithm_func(data_copy)
    end_time = time.perf_counter()

    execution_delta = end_time - start_time

    # Tuple serves as an immutable transaction snapshot of performance metrics
    self.performance_history.append((name, dataset_type, size, execution_delta))

```

5. WORKSPACE EVALUATION STATE MOCKUPS

Performance History Ledger (self.performance_history State Representation)

```

[
  ("Bubble Sort", "SORTED", 1000, 0.0002),
  ("Selection Sort", "SORTED", 1000, 0.0450),
  ("Quick Sort", "SORTED", 1000, 0.1210)
]

```

Deduplicated Domains Matrix (self.tested_sizes State Representation)

```
{1000, 5000, 10000}
```

6. LEARNING OUTCOMES

- **Structural Selection Mastery:** Evaluated algorithmic space-time tradeoffs to map runtime processing vectors to appropriate architectures rather than using arbitrary logic components.
- **Data Integrity Enforcement:** Managed logging state records securely by capturing operational execution intervals within immutable tuples to prevent accidental mutation.
- **Optimized Grouping Techniques:** Leveraged sets to capture and separate individual experimentation constraints dynamically, removing lookup loops.
- **Complex Object Aggregation:** Built deeply nested data schemas combining system metrics, dictionary lookups, and sequence arrays to control execution parameters uniformly.